

PRACTICA 2

Sistemas Operativos

Abel Martínez Molina

ÍNDICE

Análisis de la Calculadora y Conclusiones.....	3
Menú Inicial Puertas Lógicas.....	3
Actividades Puertas Logicas - Parte Visual.....	4
Actividades Puertas Logicas - Parte Funcional.....	7

Análisis de la Calculadora y Conclusiones

1. Verifica que sucede si nos situamos en el EditText que muestra el resultado y escribimos un valor después de realizar un cálculo.

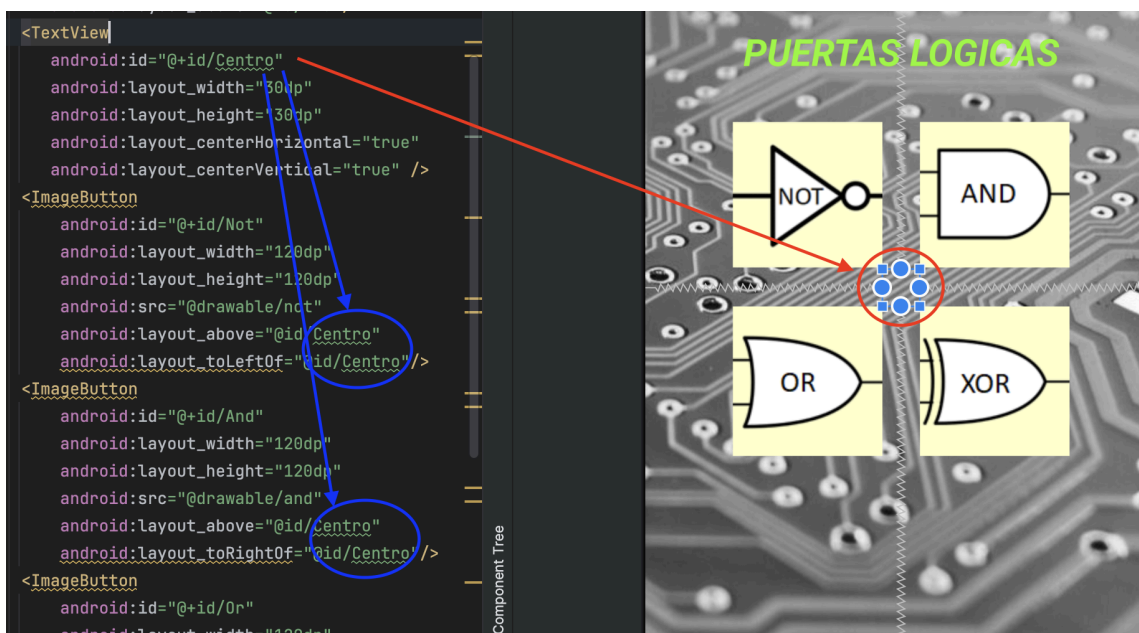
Lo que sucede es que podemos editar la casilla donde se muestran los resultados. Esto no tiene ningún sentido ya que esta solo se debería editar cuando se calcula la suma

2. Realiza las modificaciones oportunas para solucionar el problema detectado.

Para solucionarlo simplemente hay que añadir **sumaNumeros.setEnabled(false);** al principio del MainActivity.java, de forma que el usuario no pueda modificar el resultado por su cuenta.

Menú Inicial Puertas Lógicas

La creación de este menú quizás haya sido la parte más fácil del trabajo. Simplemente se compone de un **RelativeLayout** que usa un background personalizado, conformado por 4 **ImageButton** colocados respecto a un **TextView** invisible que se sitúa en el centro de la pantalla.



Para la parte funcional, el código también es bastante sencillo. Mediante el uso de **Intent** permitimos el acceso a la **activity** correspondiente para cada **ImageButton**.

```

protected void onCreate(Bundle savedInstanceState) {
    super.onCreate(savedInstanceState);
    setContentView(R.layout.activity_main);

    MediaPlayer mpok= MediaPlayer.create( context: this,R.raw.inicio);
    mpok.start();

    btn_not = findViewById(R.id.Not);
    btn_not.setOnClickListener(new View.OnClickListener() {
        public void onClick(View v) {
            Intent intent = new Intent( packageContext: MainActivity.this, MainActivity3.class);
            startActivity(intent);
        }
    });

    btn_or = findViewById(R.id.Or);
    btn_or.setOnClickListener(new View.OnClickListener() {
        public void onClick(View v) {
            Intent intent = new Intent( packageContext: MainActivity.this, MainActivity4.class);
            startActivity(intent);
        }
    });

    btn_and = findViewById(R.id.And);
    btn_and.setOnClickListener(new View.OnClickListener() {
        public void onClick(View v) {
            Intent intent = new Intent( packageContext: MainActivity.this, MainActivity2.class);
            startActivity(intent);
        }
    });
}

```

Actividades Puertas Logicas - Parte Visual

Para cada puerta lógica se ha creado un archivo.xml distinto, pero el código es prácticamente el mismo a excepción de algunas imágenes. Para explicar la estructura, nos centraremos únicamente en la activity de la puerta **AND**.

El archivo usa de estructura un **LinearLayout** vertical compuesto por dos elementos. Un **RelativeLayout** con todos los componentes funcionales, y un **ImageView** con la información de la puerta lógica AND. Ambos están distribuidos usando “gravity”. Ahora nos centraremos en los componentes del RelativeLayout:

En la parte superior del RelativeLayout tenemos de forma centrada un simple **TextView** y debajo un **RadioGroup** con 2 opciones: 1 bit y 8 bit. Ambos presentan el “checked” en false.

```

<TextView
    android:layout_height="wrap_content"
    android:text="NUMERO de BITS"
    android:layout_centerHorizontal="true"
    android:textSize="20dp"
    android:layout_marginTop="10dp"
    android:textStyle="bold"/>

<RadioGroup
    android:id="@+id/radiogroup"
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:orientation="horizontal"
    android:layout_below="@id/NumeroDeBits"
    android:layout_centerHorizontal="true">

    <RadioButton
        android:id="@+id/unbit"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:checked="false"
        android:text="1bit"/>

    <RadioButton
        android:id="@+id/ochobit"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:checked="false"
        android:text="8bit"/>

```

Luego tenemos los **EditText** que se encargan de hacer las operaciones. Tanto el EditText **A** como el **B** están dentro de un **LinearLayout** en vertical, centrado de forma vertical y alineado a la izquierda de una **ImageView** que sirve como eje central del contenedor

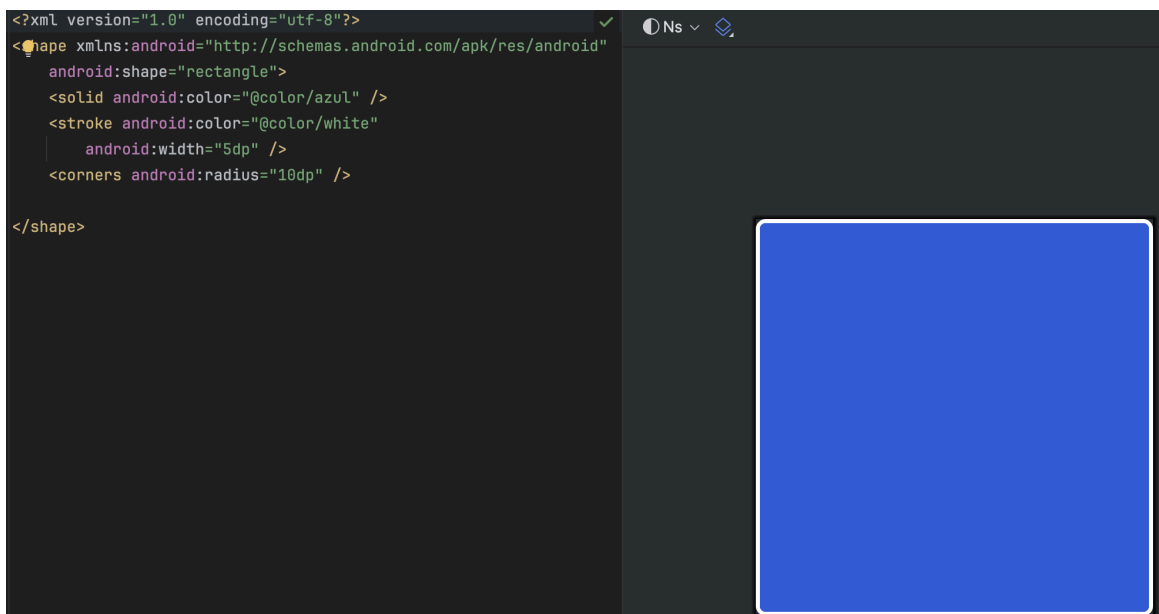
```
<LinearLayout
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:orientation="vertical"
    android:layout_centerVertical="true">

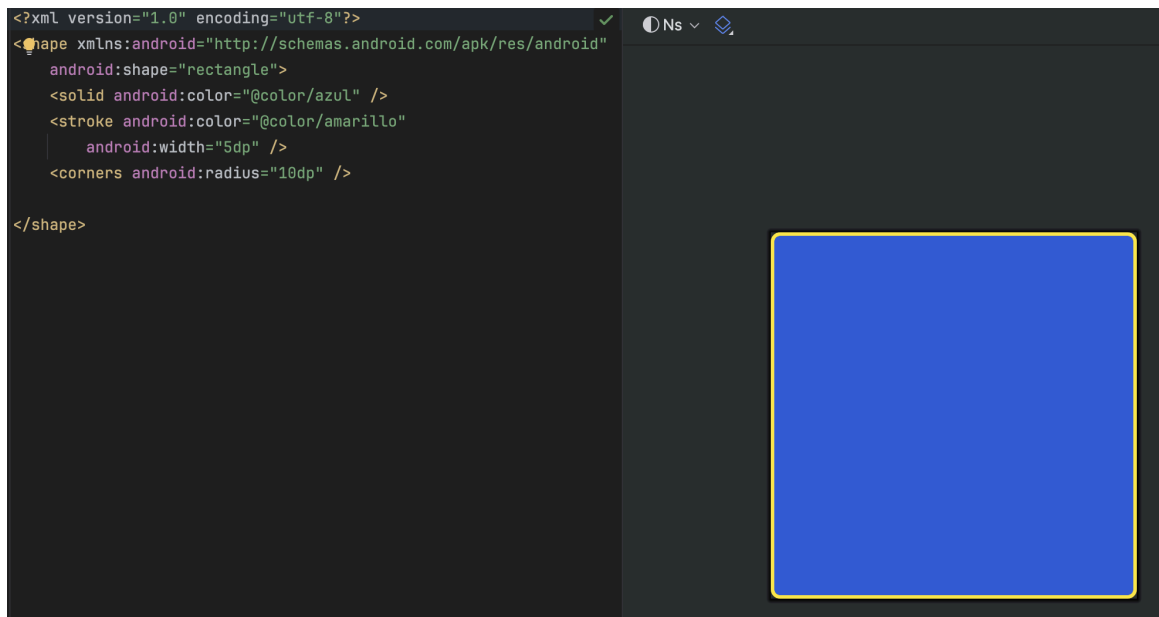
    <EditText
        android:id="@+id/a"
        android:layout_width="120dp"
        android:layout_height="45dp"
        android:background="@drawable/rectanguloazul"
        android:hint="A"
        android:textColorHint="@color/white"
        android:textColor="@color/white"
        android:textAlignment="center"
        android:layout_marginLeft="24dp"
        android:layout_toRightOf="@id/and"
        android:layout_centerVertical="true" />

    <EditText
        android:id="@+id/b"
        android:layout_width="120dp"
        android:layout_height="45dp"
        android:background="@drawable/rectanguloazul"
        android:hint="B"
        android:textColor="@color/white"
        android:textColorHint="@color/white"
        android:layout_marginLeft="24dp"
```

El EditText **C** se encuentra centrado verticalmente y alineado a la izquierda del **ImageView**. Pero en este caso no es necesario que esté contenido en un **LinearLayout**.

Estos EditText usan dos **Background** personalizados. Uno para cuando están seleccionados y otro para cuando no.

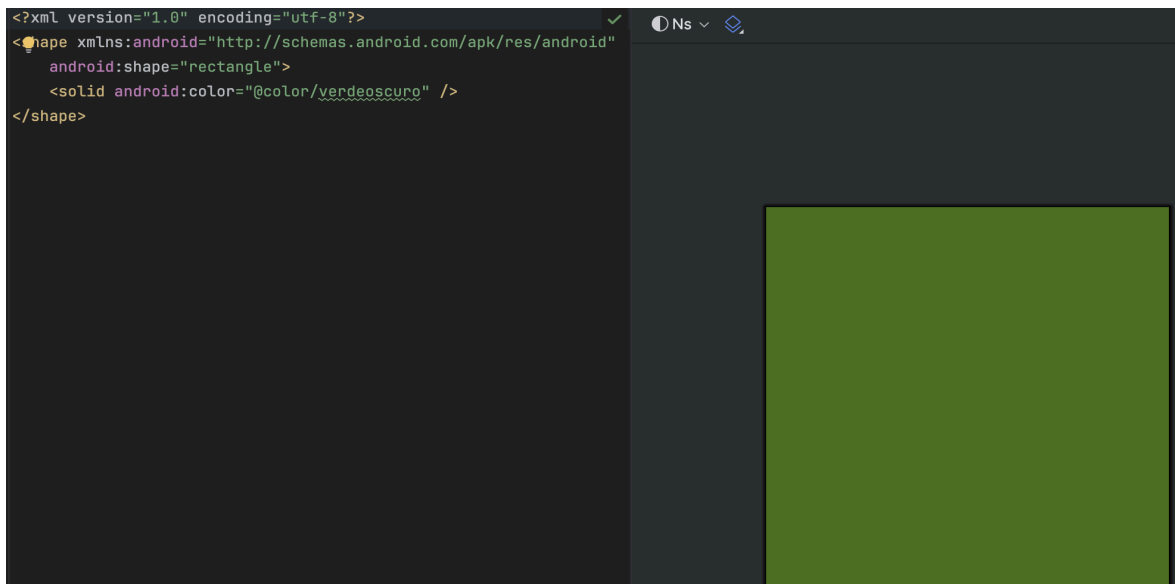




Por último se crean los botones de **Calcular**, **Borra** y **Atrás**, y se sitúan de forma centrada en la parte baja del contenedor.

El botón de Atrás es un simple **ImageButton** con una imagen de una flecha cuyo **background** ha sido modificado a un color transparente para dejar únicamente visible la imagen de la flecha sin el fondo, y donde se ha usado un **scaleType** con "center crop" para ajustar el tamaño de la imagen.

Para los otros dos botones, simplemente se ha usado un background personalizado y un **backgroundTint** para que este background personalizado sea visible



```

<Button
    android:id="@+id/calcular"
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:text="CALCULA"
    app:backgroundTint="@null"
    android:background="@drawable/boton_verde"
    android:layout_alignParentBottom="true"
    android:layout_marginBottom="20dp"
    android:layout_marginLeft="80dp"/>

<Button
    android:id="@+id/borrar"
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:text="BORRA"
    android:background="@drawable/boton_verde"
    app:backgroundTint="@null"
    android:layout_alignParentBottom="true"
    android:layout_toRightOf="@id/calcular"
    android:layout_marginBottom="20dp"
    android:layout_marginLeft="20dp"/>

<ImageButton
    android:id="@+id/flecha"
    android:layout_width="40dp"
    android:layout_height="40dp"

```

```

<ImageButton
    android:id="@+id/flecha"
    android:layout_width="40dp"
    android:layout_height="40dp"
    android:src="@drawable/back"
    android:scaleType="centerCrop"
    android:background="@color/transparente"
    android:layout_alignParentBottom="true"
    android:layout_alignParentRight="true"
    android:layout_marginBottom="20dp"
    android:layout_marginRight="10dp"/>

```

Actividades Puertas Logicas - Parte Funcional

Para cada puerta lógica se ha creado un archivo.java distinto, pero el código es prácticamente el mismo a excepción de algunas imágenes y del cálculo. Para explicar la estructura, nos centraremos únicamente en la activity de la puerta **AND**.

Cada elemento de la interfaz gráfica está declarado como una variable de instancia, inicializada dentro del método **onCreate** para mantener el control centralizado de los componentes.

```

private RadioGroup opcionGrupo;
1 usage
private RadioButton opcionUnBit, opcionOchoBit;
10 usages
private EditText campoA, campoB, campoC;
2 usages
private Button botonCalcular, botonBorrar;
2 usages
private ImageButton botonRegresar;
4 usages
private boolean modoUnBit = true; // Bandera qu

```

```

// Inicialización de las vistas (componentes de la interfaz)
opcionGrupo = findViewById(R.id.radiogroup);
opcionUnBit = findViewById(R.id.unbit);
opcionOchoBit = findViewById(R.id.ochobit);
campoA = findViewById(R.id.a);
campoB = findViewById(R.id.b);
campoC = findViewById(R.id.c);
botonCalcular = findViewById(R.id.calcular);
botonBorrar = findViewById(R.id.borrar);
botonRegresar = findViewById(R.id.flecha);

```

Uno de los principales errores que tuve es que mi código no se inicializaba correctamente debido a que uno de los 2 **RadioButton** estaba activado, así que force a que ambos estuvieran deshabilitados y el usuario tenga que seleccionar primero que **RadioButton** escoger. Si no hay ningún **RadioButton** seleccionado se debe inhabilitar la opción de escribir en los campos A y B:

```

campoA.setEnabled(false);
campoB.setEnabled(false);

```

Lo primero que se hace es configurar el comportamiento del RadioGroup con el método: **ConfigurarGrupoOpciones**. Se asigna un OnCheckedChangeListener al RadioGroup (opcionGrupo). Esto permite que el método reaccione cada vez que el usuario cambia la selección dentro del grupo de radio botones. Si este Listener se activa, el programa comprueba si la id del RadioButton activado es "unbit" si lo es se entra a la parte del código que lo configura para 1 bit, si no lo es, entra a la parte del código que se configura para 8 bits. Esta parte del código se encuentra en *configurarModo()*. Una vez que se ha configurado se borran las entradas con *borrarEntradas()* para los EditText A y B, y se llama a un módulo *habilitarEntradas()* que permite escribir en los EditText

```
private void configurarGrupoOpciones() {
    opcionGrupo.setOnCheckedChangeListener(new RadioGroup.OnCheckedChangeListener()
    {
        @Override
        public void onCheckedChanged(RadioGroup group, int checkedId) {
            // Verifica qué opción fue seleccionada y ajusta el modo de operación
            modoUnBit = (checkedId == R.id.unbit);
            configurarModo(); // Configura el modo según la selección
        }
    });
}
```

```
private void configurarModo() {
    if (modoUnBit) {
        // Modo de 1 bit: permite solo un carácter en los campos de entrada
        campoA.setFilters(new android.text.InputFilter[]{new android.text.InputFilter.
        LengthFilter(1)});
        campoB.setFilters(new android.text.InputFilter[]{new android.text.InputFilter.
        LengthFilter(1)});
    } else {
        // Modo de 8 bits: permite hasta 8 caracteres en los campos de entrada
        campoA.setFilters(new android.text.InputFilter[]{new android.text.InputFilter.
        LengthFilter(8)});
        campoB.setFilters(new android.text.InputFilter[]{new android.text.InputFilter.
        LengthFilter(8)});
    }
    borrarEntradas(); // Limpia los campos después de cambiar el modo
}
```

```
private void habilitarEntradas() {
    campoA.setEnabled(true); // Habilita el campo A
    campoB.setEnabled(true); // Habilita el campo B
}
```

```
private void borrarEntradas() {
    campoA.setText(""); // Limpia el campo A
    campoB.setText(""); // Limpia el campo B
    campoC.setText(""); // Limpia el campo de salida
}
```

Una vez configurado el RadioGroup ya se pueden calcular las operaciones lógicas por lo que ya se puede hacer uso del botón calcular. Sin embargo, este botón está diseñado para que solo funcione cuando se han rellenado los campos de forma correcta. Esto se programa en el método *calcularResultado()*:

Lo primero que se hace es obtener los valores de los campos A y B y los transforma en **String**. Luego verifica si cumple el primer requisito “**Que los campos no estén vacíos**”. Si este requisito no se cumple, muestra una notificación y un sonido de error:

```
// Obtiene los valores de entrada, eliminando espacios en blanco
String valorA = campoA.getText().toString().trim();
String valorB = campoB.getText().toString().trim();

// Verifica si los campos están vacíos
if (valorA.isEmpty() || valorB.isEmpty()) {
    Toast.makeText( context: this, text: "Por favor ingresa los valores A y B", Toast.LENGTH_SHORT).show();
    MediaPlayer sonidoError = MediaPlayer.create( context: this, R.raw.nok); // Reproduce un sonido de error
    sonidoError.start();
    return;
}
```

Luego verifica que se cumple el segundo requisito “**Que los valores de entrada solo sean 0 o 1**” con el método *validarEntrada()*. Si este requisito no se cumple muestra una notificación y un sonido de error.

```
// Valida que las entradas contengan solo los valores permitidos (0 o 1)
if (!validarEntrada(valorA) || !validarEntrada(valorB)) {
    Toast.makeText( context: this, text: "Solo se permiten valores 0 o 1", Toast.LENGTH_SHORT).show();
    MediaPlayer sonidoError = MediaPlayer.create( context: this, R.raw.nok); // Reproduce un sonido de error
    sonidoError.start();
    return;
}
```

```
private boolean validarEntrada(String entrada) {
    return entrada.matches( regex: "[01]+" ); // Veri
}
```

El programa está diseñado para que no se puedan escribir números más grandes que 1 en un caso y 8 en el otro caso, ya que así se configura en *configurarModo()*. Pero en el caso particular de los 8 bits, si se escriben menos de 8 números, debemos modificar el input para que se rellenen los números restantes para llegar a 8 con 0 usando *replace*:

```
if (!modoUnBit) {
    if (valorA.length() < 8) {
        valorA = String.format("%-8s", valorA).replace( oldChar: ' ', newChar: '0'); // Completa con ceros
        campoA.setText(valorA); // Actualiza el campo A
    }
    if (valorB.length() < 8) {
        valorB = String.format("%-8s", valorB).replace( oldChar: ' ', newChar: '0'); // Completa con ceros
        campoB.setText(valorB); // Actualiza el campo B
    }
}
```

Si todos los requisitos están verificados, se realiza la operación lógica, en este caso, la propia de una puerta AND. Para el modo de un bit se calcula el resultado de forma simple ya que solo hay que hacer la operación entre 2 términos. Pero para el modo de 8 bits se usa un bucle que calcula

cada término con cada término y almacena el resultado en un `StringBuilder`. Cuando la operación se completa, suena un sonido de éxito.

```
// Realiza la operación lógica AND según el modo seleccionado
if (modoUnBit) {
    // Operación de 1 bit
    int bitA = Integer.parseInt(valorA);
    int bitB = Integer.parseInt(valorB);
    int resultado = bitA & bitB; // AND lógico
    campoC.setText(String.valueOf(resultado)); // Muestra el resultado
    MediaPlayer sonidoExito = MediaPlayer.create(context: this, R.raw.ok); // Reproduce un sonido de éxito
    sonidoExito.start();
} else {
    // Operación de 8 bits
    StringBuilder resultado = new StringBuilder();
    for (int i = 0; i < 8; i++) {
        int bitA = Character.getNumericValue(valorA.charAt(i));
        int bitB = Character.getNumericValue(valorB.charAt(i));
        resultado.append(bitA & bitB); // Calcula el AND de cada par de bits
    }
    campoC.setText(resultado.toString()); // Muestra el resultado
    MediaPlayer sonidoExito = MediaPlayer.create(context: this, R.raw.ok); // Reproduce un sonido de éxito
    sonidoExito.start();
}
```

También se configuran los bordes de los `EditText` A y B gracias a un `OnFocusChangeListener` que se usa en el método `configuraFocusChangeListener()`.

```
private void configuraFocusChangeListener() {
    View.OnFocusChangeListener focusChangeListener = new View.OnFocusChangeListener() {
        @Override
        public void onFocusChange(View v, boolean hasFocus) {
            if (hasFocus) {
                v.setBackgroundResource(R.drawable.bordesactivos); // Cambia el fondo al personalizado
            } else {
                v.setBackgroundResource(R.drawable.rectanguloazul); // Restaura el fondo predeterminado
            }
        }
    };

    campoA.setOnFocusChangeListener(focusChangeListener);
    campoB.setOnFocusChangeListener(focusChangeListener);
}
```

Por último, el botón Borrar se configura llamando al método `ClearInputs()` que limpia los campos A, B y C. Y la flecha para volver atrás se configura mediante un `Intent` que lleva al menú principal

```
// Configurar el botón "Borrar"
botonBorrar.setOnClickListener(new View.OnClickListener() {
    @Override
    public void onClick(View v) {
        clearInputs();
    }
});

botonRegresar.setOnClickListener(new View.OnClickListener() {
    public void onClick(View v) {
        Intent intent = new Intent(packageContext: MainActivity5.this, MainActivity.class);
        startActivity(intent);
    }
});
```